

Famto Backend

Microservice Architecture

Migration Plan

Prepared: June 01, 2026 | Team: 2 Developers | Timeline: 52 Weeks

Current State	Target State
22,500 lines monolith	8 independent microservices
65 controllers, 79 models	Domain-driven service boundaries
Single process, no caching	Docker + Kubernetes + Redis
No message queue	Kafka + RabbitMQ event-driven
268+ console.log calls	Structured Pino logging
Zero automated tests	Integration tests per service

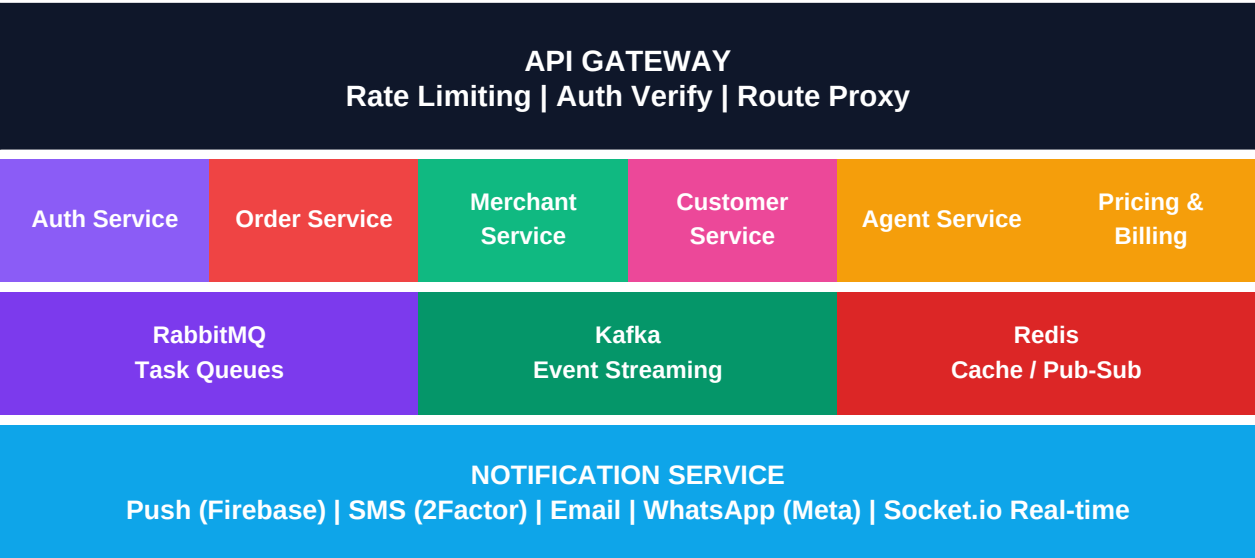
Contents

1. Architecture Overview
2. Service Ownership Matrix
3. Phase 0: Foundation Hardening (Weeks 1-6)
4. Phase 1: Notification Service (Weeks 7-12)
5. Phase 2: Auth Service (Weeks 13-16)
6. Phase 3: Pricing & Billing Service (Weeks 17-22)
7. Phase 4: Merchant Service (Weeks 23-28)
8. Phase 5: Customer + Agent Services (Weeks 29-36)
9. Phase 6: Order Service + Kafka (Weeks 37-46)
10. Phase 7: Kubernetes + Decommission (Weeks 47-52)
11. Data Migration Strategy
12. Technology Stack
13. Developer Week-by-Week Allocation
14. Risk Mitigations
15. Timeline & Recommendation
16. Verification Checklist

1. Architecture Overview

The migration follows the **Strangler Fig** pattern: services are extracted one at a time behind an API Gateway. The frontend sees zero URL changes (except Socket.io connection URL, once). Production traffic is never interrupted.

Target: 8 Microservices



2. Service Ownership Matrix

Service	Models Owned	Key Controllers
Auth	Admin, Manager, ManagerRoles, Token	authController
Order	Order, TemporaryOrder, ScheduledOrder, Task, PickAndCustomCart, DatabaseCounter	adminOrderController, merchantOrderController, universalOrderController, pickAndDropController
Merchant	Merchant, Category, Product, BusinessCategory, Discounts, Banners	merchantController, categoryController, productController
Customer	Customer, CustomerCart, Wallet, LoyaltyPoint, Referral, PromoCode	customerController
Agent	Agent, AgentTransaction, AgentWorkHistory, AutoAllocation	agentController
Pricing & Billing	CustomerPricing, MerchantPricing, AgentPricing, *Surge, Tax, Commission, Subscription	pricingControllers, subscriptionController, homeController
Notification	NotificationSettings, FCMTOKEN, all NotificationLogs, all Whatsapp* (8 models)	pushNotificationController, whatsapp/*, socket.js
API Gateway	None (stateless)	Route proxying, rate limiting, JWT verification

PHASE 0 Foundation Hardening (Weeks 1-6)

No service split yet. Harden the monolith with structured logging, Redis caching, Docker Compose, database optimization, and graceful shutdown. This phase prevents cascading problems in every subsequent phase.

Week 1-2: Logging + Error Handling + Graceful Shutdown

Dev A - Structured Logging

- Install Pino (pino + pino-http). Create utils/logger.js shared module
- Replace all 268+ console.log/console.error calls with logger.info/logger.error
- Add pino-http middleware to index.js for request-level logging with request-id correlation

Dev B - Error Handling + Shutdown

- Enhance middlewares/globalErrorHandler.js: distinguish 4xx vs 5xx, log stack traces
- Add process.on('unhandledRejection') and process.on('uncaughtException') handlers
- Implement graceful shutdown: SIGTERM/SIGINT -> drain Socket.io -> close MongoDB -> exit

Week 3-4: Redis + Docker Compose

Dev A - Redis Integration

- Add Redis to stack. Create config/redis.js connection module
- Replace in-memory distanceCache with Redis (TTL: 5 min)
- Replace node-cache in isAuthenticated.js with Redis
- Add Redis-based rate limiting (express-rate-limit + rate-limit-redis)

Dev B - Docker Compose + Firebase Refactor

- Create docker-compose.yml with: app, MongoDB (local dev), Redis
- Add GET /health endpoint returning MongoDB + Redis connection status
- Move Firebase Admin SDK init from socket.js into shared config/firebaseAdmin.js

Week 5-6: Database Optimization + Service Layer

Dev A - DB Performance

- Audit all 79 models for missing indexes (Order, Merchant, Task are critical)
- Add .lean() to all read-only queries in top-5 controllers
- Fix N+1 patterns: merge sequential countDocuments into single aggregation

Dev B - Service Layer Template

- Create services/ and repositories/ directories

- Refactor geofenceController (155 lines) and customerPricingController (305 lines) as templates
- Document Controller -> Service -> Repository pattern in CONTRIBUTING.md

Deliverable: Structured logging, Redis caching, Docker Compose dev setup, health checks, DB indexes, graceful shutdown. Monolith is production-hardened.

PHASE 1 Extract Notification Service (Weeks 7-12)

Why first: socket.js is imported by 20+ files and is the biggest coupling point. Notification logic has clear inputs (userId, event, data) and outputs (push/socket/log). Extracting it decouples every subsequent extraction.

Week 7-8: Service Scaffold + RabbitMQ

Dev A

- Create services/notification-service/ project (separate package.json, Dockerfile)
- Add RabbitMQ to docker-compose.yml
- Define queues: notification.push, notification.socket, notification.sms, notification.email, notification.log

Dev B

- Extract sendPushNotificationToUser(), createNotificationLog(), sendNotification(), sendSocketData()
- Move Firebase Admin SDK config to Notification Service
- Create REST API endpoints for direct notification sends

Week 9-10: Event-Driven Integration

Dev A

- Replace direct sendNotification()/sendSocketData() calls (20 files) with RabbitMQ publishers

Dev B

- Migrate Socket.io server to Notification Service with Redis adapter for horizontal scaling
- Monolith publishes events -> RabbitMQ -> Notification Service -> Socket.io emit

Week 11-12: WhatsApp Module + Testing

Dev A

- Move all 9 WhatsApp controllers + 8 models into Notification Service

Dev B

- Integration test all 22 socket events end-to-end
- Verify push notifications to both Firebase projects
- Load test RabbitMQ throughput

Deliverable: Notification Service running in Docker Compose. Monolith no longer contains socket.js notification logic. RabbitMQ connects them.

PHASE 2 Extract Auth Service (Weeks 13-16)

Week 13-14: Auth Service

Dev A

- Create Auth Service: POST /auth/login (Admin/Merchant/Manager), POST /auth/otp/send + /verify (Customer/Agent), POST /auth/register, POST /auth/refresh-token

Dev B

- Create JWT validation as shared middleware (verify signature locally without calling Auth Service per request)
- Redis for token blacklisting (logout invalidation)

Week 15-16: API Gateway

Dev A

- Set up Express-based API Gateway using http-proxy-middleware
- Route /api/v1/auth/* -> Auth Service, /api/v1/whatsapp/* -> Notification Service, everything else -> monolith

Dev B

- Remove auth controllers from monolith. Update isAuthenticated.js to verify JWT locally
- Test all 5 actor login flows end-to-end

Deliverable: Auth Service + API Gateway running. All traffic flows through gateway.

PHASE 3 Extract Pricing & Billing Service (Weeks 17-22)

Week 17-20: Move 6 pricing controllers, 6 surge controllers, Tax, Commission, Subscription controllers and models. Expose internal API: GET /pricing/calculate?geofenceId=X&distance=Y&vehicleType=Z. Move billing cron jobs from index.js.

Week 21-22: Move homeController.js dashboard aggregations, createPerDayRevenueHelper.js, revenue models. Verify pricing calculations match monolith exactly.

Deliverable: Pricing Service running. Monolith calls Pricing API for all calculations.

PHASE 4 Extract Merchant Service (Weeks 23-28)

Move Merchant, Category, Product, BusinessCategory, Discounts, Banners. Expose internal APIs for Order Service: GET /merchants/:id, GET /products/:id, PATCH /products/:id/stock. Publish Kafka events: merchant.status.changed, product.stock.updated. Move payout logic and merchant cron jobs.

Deliverable: Merchant catalog served from Merchant Service.

PHASE 5 Extract Customer + Agent Services (Weeks 29-36)

Weeks 29-31 (Customer): Move Customer model, cart, wallet, loyalty, referral, promo codes. Expose GET /customers/:id/validate, POST /customers/:id/wallet/debit.

Weeks 32-34 (Agent): Move Agent model, transactions, work history, location tracking, auto-allocation. Expose GET /agents/available?geofenceId=X.

Weeks 35-36: Full integration testing across all extracted services.

Deliverable: Customer + Agent Services running independently.

PHASE 6 Extract Order Service + Kafka (Weeks 37-46)

The hardest phase. Order domain is ~15,000 lines with deep coupling to every other service.

Week 37-38: Kafka Setup

Add Kafka to infrastructure. Define events: order.created, order.accepted, order.assigned, order.pickup.started, order.delivered, order.cancelled, order.payment.completed.

Week 39-42: Order Service Core

Move Order, TemporaryOrder, ScheduledOrder, Task models. Move ProcessOrderService.js, createOrderHelpers.js, orderCreateTaskHelper.js. Replace all direct model imports with inter-service HTTP calls.

Week 43-44: Payment + Socket Events

Move Razorpay integration. Move remaining order-related socket events. Order Service publishes to Kafka; Notification Service consumes and dispatches.

Week 45-46: Cron Migration + E2E Testing

Full lifecycle test: Create -> Pay -> Assign -> Pickup -> Deliver -> Complete. Test all 4 delivery modes. Test scheduled orders. Test payment failure/refund paths.

Deliverable: Order Service is source of truth for all orders. Monolith is now the gateway.

PHASE 7 Kubernetes + Decommission (Weeks 47-52)

Week 47-48: Remove all extracted code from monolith. Remaining admin operations become thin Admin Service.

Week 49-50: Create K8s manifests (Deployments, Services, ConfigMaps, Secrets, HPA). Set up Helm charts. CI/CD pipeline with GitHub Actions.

Week 51-52: Blue-green deployment to K8s cluster. Old monolith as fallback for 2 weeks.

Deliverable: All 8 services on Kubernetes. Monolith decommissioned.

11. Data Migration Strategy

Phase	Strategy	Risk Level
Phase 0-2	All services share same MongoDB Atlas cluster + database. No data migration.	Low
Phase 3-5	Each new service gets own database on same Atlas cluster. One-time migration script. Brief read-only cutover (minutes).	Medium
Phase 6+	Each service owns its data. Cross-service reads via API calls. Eventual consistency via Kafka events.	Medium

12. Technology Stack Per Service

```
src/config/ (DB, Redis, Kafka, RabbitMQ) | src/controllers/ (thin HTTP handlers) |
src/services/ (business logic) | src/repositories/ (Mongoose data access) |
src/middlewares/ (auth, validation) | src/events/ (Kafka) | src/queues/ (RabbitMQ) |
src/utils/
```

Technology	Purpose
Pino	Structured JSON logging with request-id correlation
Joi / Zod	Request validation (replaces express-validator)
Redis	Caching + rate limiting + session + Socket.io adapter
RabbitMQ	Task queues (notifications, campaigns, async jobs)
Kafka	Event streaming (order lifecycle, merchant status updates)
Docker + K8s	Containerization + orchestration + HPA autoscaling
GitHub Actions	CI/CD pipeline per service

13. Developer Week-by-Week Allocation

Week	Dev A	Dev B
1-2	Structured logging (Pino)	Graceful shutdown, error handling
3-4	Redis, caching, rate limiting	Docker Compose, Firebase refactor, health check
5-6	DB indexes, lean queries, N+1 fixes	Service layer pattern, refactor 2 controllers
7-8	Notification Service scaffold, RabbitMQ	Extract notification functions from socket.js
9-10	RabbitMQ producers in monolith	Socket.io migration to Notification Service
11-12	WhatsApp module migration	Integration testing all socket events
13-14	Auth Service implementation	JWT shared validation, Redis token blacklist
15-16	API Gateway setup	Monolith auth removal, testing
17-20	Pricing Service core	Internal pricing API + subscription migration
21-22	Revenue/analytics migration	Integration testing pricing
23-26	Merchant profiles + products + APIs	Discounts, banners, Kafka events
27-28	Payout migration	End-to-end testing
29-31	Customer Service core	Customer controllers migration
32-34	Agent Service core	Agent controllers migration
35-36	Integration testing	Integration testing
37-38	Kafka setup, event schema	Order flow refactor to service pattern
39-42	Order Service core models	Inter-service call implementation
43-44	Socket order events migration	Payment/Razorpay migration
45-46	Cron migration	E2E order lifecycle testing
47-48	Monolith cleanup	Admin Service remainder
49-50	Kubernetes manifests + Helm	CI/CD pipeline
51-52	Production deployment	Monitoring + fallback

14. Risk Mitigations

Risk	Likelihood	Impact	Mitigation
Socket.io migration breaks real-time	High	Critical	Feature flag to toggle old/new. Keep monolith Socket.io as fallback for 2 weeks
Order race conditions during split	High	Critical	Redis distributed locks + MongoDB transactions. Never have two services writing to same order
Team burnout / scope creep	High	High	Strict phase gates. No new phase until previous is stable in production for 1 week
Data inconsistency across services	Medium	Critical	Start with shared DB. Move to separate DBs only when boundary is proven. Kafka dead-letter queues
Inter-service HTTP latency	Medium	Medium	Redis caching for hot data (pricing, merchant details). Connection pooling. Batch API calls
Kafka/RabbitMQ operational complexity	Medium	Medium	Start with RabbitMQ only. Add Kafka in Phase 6 when event ordering matters
Custom ID generation (DatabaseCounter) conflicts	Low	Critical	Keep DatabaseCounter in single service (Order). Uses findOneAndUpdate with \$inc (atomic)

15. Timeline & Recommendation

Scenario	Duration
With zero feature development	52 weeks (12 months)
With feature work interleaved	15-18 months
Phase 0 + Phase 1 only (80/20 rule)	12 weeks (3 months)

80/20 Recommendation: Phase 0 + Phase 1 alone (12 weeks) delivers 80% of the architectural improvement. Structured logging, Redis, Docker Compose, graceful shutdown, and decoupled notifications make the system dramatically more reliable. Treat Phase 2-7 as optional based on actual scaling needs.

16. Verification Checklist

After each phase, verify:

- Run integration tests against extracted service (supertest / Newman collections)
- Compare API responses with monolith (same inputs -> same outputs)
- Monitor error rates in production for 1 week before starting next phase
- Verify all frontend flows (Famto Dashboard + mobile apps) work identically
- Load test the new service with production-like traffic patterns
- Verify RabbitMQ / Kafka message delivery with no dropped events
- Check Redis cache hit rates and memory usage
- Confirm zero data inconsistency between services

End of Document | Famto Microservice Migration Plan | Confidential